Evicting Makes It Slower: Three Properties a KV Cache Policy Needs on a Phone

Md Romyull Islam
Kennesaw State University
Marietta, Georgia, USA
romyullislam2012@gmail.com

ABSTRACT

On a phone the KV cache, not compute, limits sustained decode. We ran five published eviction policies on a OnePlus 15 inside `llama.cpp` at their own published budgets. None behaves as published. Their budgets do not materialize, because one cell array serves every head and layer: a per-head budget leaves 36 to 68% of the cache live, against the 5 to 10 times reduction the papers report. Their scores cost the fast path, because reading post-softmax attention leaves the fused FlashAttention kernel. Every score-reading policy decodes at 0.12 to 0.20 times the full cache. On this phone, evicting is slower than not evicting.

Three properties decide the outcome, and every published policy lacks one. Selection must be sequence-level, or no cell is freed. It must not read scores at decode, or the kernel is lost. It must be content-aware, or retrieval fails. StreamingLLM has the first two and answers 3 of 14 needles. The four per-head policies have the third and lose the other two.

μKV is the existence proof that one policy can hold all three. It computes real attention scores inside the FlashAttention prefill graph through a small side node, selects one keep set for every head and layer, and compacts in place. On the Adreno 840 it decodes at 1.19 times the full cache on 7.4% of the cells. A corrected StreamingLLM ties that speed, which is our strongest evidence that the mechanism is real, but it needs three times the budget and loses the needles. μKV ties the full cache across a 392-cell needle sweep and stays within 0.1 to 3.7 F1 on LongBench at 13% of the cache. On the CPU it decodes 4.8 times faster than the full cache at 62% less energy. We argue that on-device cache work should report realized cells, not selection ratios.

KEYWORDS

Mobile LLM inference, KV cache eviction, FlashAttention, thermal management, energy-aware scheduling, on-device AI

1 INTRODUCTION

Small instruction-tuned models now run interactively on flagship phones. For a sustained generation the wall is the KV cache, not the matrix multiply. Every decode step reads the whole cache once per layer, so past a thousand tokens the step is bound by DRAM bandwidth. The DRAM heats, the vendor thermal engine cuts the clock, and the battery drains. Cache size sets traffic, traffic sets power, power sets temperature, temperature sets the throttle, and the throttle sets throughput.

Five policies bound that cache in the literature: H2O [1], TOVA [3], SnapKV [4], StreamingLLM [2] and Ada-KV [6]. All were built for server GPUs and judged by perplexity alone. We ran each in its own published configuration on a OnePlus 15 inside `llama.cpp`, across four models, measuring quality, throughput, heat and energy together. None behaves the way its paper reports. Two engine properties explain it, and neither is visible from a server.

- **Budgets do not materialize.** One cell array serves every head and layer, so a cell is freed only when every head and every layer drops it. SnapKV reports an 8.2 times better memory efficiency at 16K inputs [4]; at its own budget on the phone it keeps 94% of the cells. H2O reports 5 to 10 times at a 20% budget [1]; at that budget it keeps 92%. Across the four per-head policies, 36 to 68% of the cache stays live at a nominal 1024.
- **Scores cost the fast path.** A policy that reads post-softmax attention cannot use the fused FlashAttention kernel [10]. Every score-reading policy decodes at 0.12 to 0.20 times the full cache on the phone GPU. Evicting is slower than not evicting.

Those two properties, with retrieval quality, give three requirements. Selection must be sequence-level, or no cell is freed. It must not read scores at decode, or the kernel is lost. It must be content-aware, or retrieval fails. Every published policy fails one of them. StreamingLLM meets the first two and answers 3 of 14 needles. The four per-head policies meet the third and fail the other two.

μKV is the existence proof that one policy can hold all three at once (Figure 1). It computes real attention scores inside the FlashAttention prefill graph through a small side node, so content awareness costs 1.9% of prefill and no decode bandwidth. It selects one keep set of K cells for every head and layer, so eviction frees cells. It compacts in place, so survivors slide into a dense prefix inside the tensors prefill already allocated. This paper makes five contributions.

(i) A measurement no server study can produce: five published policies at their own budgets, on a phone, on quality, throughput, heat and energy at once (§4).
(ii) The mechanism behind the gap, with controls that isolate each property: disabling compaction, and forcing a policy off the fused kernel (§4).
(iii) μKV, and the finding that a corrected StreamingLLM, which we did not author, reaches the same throughput and so confirms the mechanism is not ours (§3, §4).
(iv) Evidence that the gap follows from how KV memory is laid out on devices, not from one engine (§4).
(v) An evaluation practice for on-device cache work: report realized cells, not selection ratios (§6).

2 BACKGROUND

H2O [1] keeps heavy hitters by accumulated attention plus recent tokens and reports a 5 to 10 times smaller cache at a 20% budget.

TOVA [3] drops the least-attended position per layer every step and reports results nearly on par with the full model at, in some cases, one eighth of the cache. SnapKV [4] freezes a per-head top-K at the end of prefill and reports an 8.2 times better memory efficiency at 16K inputs. StreamingLLM [2] keeps 4 sinks and a sliding window with no scores and states that it adds no long-term memory. Ada-KV [6] allocates one layer budget across heads and needs FlashAttention-2's variable-length kernel [11] plus a custom CUDA kernel, both absent on the mobile stack. FlashAttention [10] fuses softmax into the kernel and never writes the attention tensor, so a policy that reads scores is locked to the slow path. `llama.cpp` [14], like every on-device engine we know of, stores the cache as one array of position-indexed cells shared across heads and layers. It removes cells by position and compacts through a state API that restores into a second context. Neither fact is visible on a server with per-head paged storage.

On the control side, zTT [20] and FUSE [21] govern frequency from temperature or phase, with no cache signal. The vendor engine on our SoC deep-throttles the prime cores to 883 MHz the moment the battery reaches 50 ℃, and caps the GPU at 726 MHz when DDR passes 64 ℃. Both are single cliffs. μKV's watchdog descends a ladder anchored above each cliff, so the vendor step is never reached (Figure 1).

3 μKV DESIGN

Scoring inside the FlashAttention graph. The fused kernel never materializes attention. μKV adds a `kq_evict` side node to the prefill graph instead of leaving the fast path. The node computes $\mathrm{softmax}(KQ^\top)$ for the last 16 queries of the last prefill chunk and reduces it to one score per layer, head and position, $\bar{A}_{l,h,p}$. It is a graph output, so the mid-graph interception that corrupts this Vulkan backend cannot touch it. It costs 1.2% of prefill on the CPU (paired, n=29) and 138 against 131 s on the GPU. Selection is bit-identical to a FlashAttention-off capture.

One keep set for all heads and layers. Selection is sequence-level. The position score is the mean of $\bar{A}_{l,h,p}$ over layers and heads, max-pooled over 7 positions so a run of related tokens survives together (SnapKV's clustering step). A per-prompt split α_a, the fraction of attention mass outside the recent window (floored at 0.70, capped at 0.95), sets how many of the K cells are distant anchors; the rest is the recent window. The top n_{anchor} positions by pooled score survive, plus four sink tokens [2]. The result is one set of K positions shared by every head and layer, which is what lets the cache compact. On the 9737-token prompt $\alpha_a = 0.71$, so K=1024 is 4 sinks, 721 anchors and 299 recent cells. Earlier versions also sized a per-head budget from each head's peak attention through a closed-form ramp. Its candidate pool holds 96.5 to 99.8% of the prompt at K=1024 on every cell. Bypassing it leaves the keep set identical (721 of 721 positions) and the output identical, so it is removed. K itself is sized from the request's demand, the larger of a retrieval floor and a generation floor. In a controlled test the budget that still retrieves a fact scales with the generated span, from 256 cells at 64 tokens to 4096 at 2048, not with the prompt.

In-place compaction and decode. Compaction through the state API doubles peak cache memory at the instant of restore. Phi-3-mini at 16K needs 2×6144 MiB plus 2.4 GB of weights, and Android killed the process and six co-resident apps, twice. `endurkv_compact_seq()` instead slides survivors into a dense prefix in bounded chunks inside the tensors prefill already allocated. Peak memory does not rise, the configuration runs at 878 cells, and the two modes produce identical keep sets (perplexities agree to 0.15% across four model families). Decode runs over the K-cell cache with the fused kernel. The keep set stays frozen while the recent window slides. A re-eviction fires when the cache's position span passes 1.25 times the prompt plus the window; on a 4096-token decode it fires once, and the decode-mean live cache is 1980.

Reduce-only clock watchdog. A daemon samples skin, battery and DDR temperature at 2 Hz and drives three caps: one per CPU cluster and one for the GPU. It lowers a cap one step at a time and never raises it. Each ladder is anchored at a measured vendor trigger. The CPU ladders have five steps, 1497/1382/1267/1132/1017 MHz, and two sensors reach them independently: battery at 47.0/48.0/48.5/49.0/49.5 ℃ and skin at the same staircase offset 3 ℃ higher. Each cluster takes the lower of the two, so whichever sensor heats first governs. The floor stays above the 883 MHz cliff the vendor applies the instant the battery reaches 50 ℃, so the watchdog never self-throttles, and DDR above 71 ℃ floors both clusters to 1267 as a backstop. The GPU ladder has three steps on DDR alone, 1050, 967 and 902 MHz at 60, 62 and 63.5 ℃, with 1.5 ℃ of hysteresis on the way back up and below the vendor's 726 MHz trip at 64 ℃. The kernel applies the lower of the user cap and the vendor cap, so the ladders act earlier than the vendor and never later. On a cold phone every ladder stays dormant.

4 EVALUATION

Setup. OnePlus 15: Snapdragon 8 Elite Gen 5, 16 GB, Adreno 840. Models: Llama-3.2-1B, Phi-3-mini-128k and gemma-2-2b-it in Q4_K_M, and PrismML's 1-bit Bonsai-8B [24]. Every timed cell starts behind a cooling gate (DDR $\leq 35\,C$, battery $\leq 33\,C$) with charging off, on one build, under the vendor's own DVFS and thermal engine. Baselines run their own papers' selection rules. The tables use a matched budget of 1024. SnapKV and Ada-KV keep 1024 cells per head (window 64, kernel 5; Ada-KV safeguard 0.2). H2O keeps 1024 per head, split evenly between heavy hitters and recent tokens. TOVA keeps 1024 per layer with 4 sinks. StreamingLLM keeps 4 sinks and 1020 recent cells. A second campaign runs every baseline at its own published budget (below). Energy integrates the USB rail and the coulomb counter, split at the prefill and decode boundary. μKV runs with K=1024 and the watchdog throughout.

Sustained decode on the CPU.. Table 1 runs a 9.7K-token prompt followed by 4096 generated tokens. Every policy builds the full prompt cache during prefill (Figure 2). μKV then compacts to 721 cells, and its re-eviction keeps the cache bounded near 2K. SnapKV can only drop to 5.9K, and vanilla grows without bound. On Llama-3.2-1B the bounded cache buys 24.2 against 5.0 tok/s, 2.6 times less wall time and 62% less energy. Its DDR peak is 5 ℃ below vanilla and level with TOVA's, at the same vendor-clamped 1.6 GHz. SnapKV, Ada-KV, H2O and TOVA keep 3.5K to 6.1K cells live at a nominal 1024. On Llama-3.2-1B that is a 1.6 to 2.8

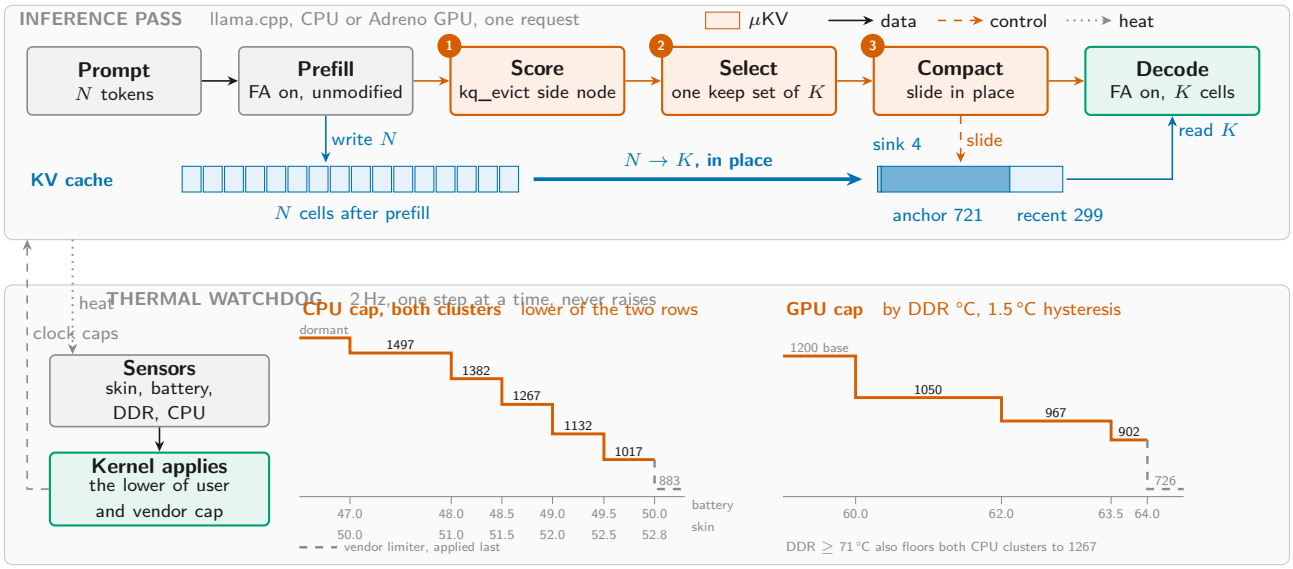

Figure 1: μKV on the phone. Top: the inference pass, where scoring, selection and compaction (1 to 3) are the only changes. Bottom: the watchdog, which lowers each clock cap one step at a time along a ladder anchored above the vendor cliff.

Table 1: Sustained decode on the CPU, cold start. One text, retokenized per model: 9737, 10074, 7542 and 6382 tokens. Upper blocks 4096 generated at K=1024, lower blocks 2048 at each policy's own budget.

Policy	Budget	tok/s	Wall (s)	Live cells	DDR (°C)	mWh
Llama-3.2-1B						
vanilla (full cache)	1024	5.0	1055	9737	59.8	1033
μKV	1024	**24.2**	**406**	721 (1980)	**54.4**	**391**
SnapKV	1024	6.8	902	5929	56.3	818
Ada-KV	1024	6.7	875	3484	57.1	1002
StreamingLLM	1024	6.6	877	777	56.7	1052
H2O	1024	5.8	1006	6110	54.8	1094
TOVA	1024	6.0	947	4091	54.4	1069
Bonsai-8B, 1-bit weights						
vanilla (full cache)	1024	1.0	5137	10074	**72.2**	5758
μKV	1024	**4.5**	**2012**	789 (1993)	**62.5**	**2312**
SnapKV	1024	1.1	4713	10015	67.9	5476
Ada-KV	1024	1.6	3718	6514	65.2	3971
StreamingLLM	1024	1.6	3777	858	63.7	4012
H2O	1024	1.5	3900	9587	63.3	4136
TOVA	1024	1.6	3841	7116	67.2	4581
Phi-3-mini, 7542-token prompt						
vanilla (full cache)	1024	1.2	2063	7542	60.6	2046
μKV	1024	**5.5**	**841**	929	**56.3**	**926**
SnapKV	1024/head	1.7	1980	7130	66.0	2530
Ada-KV	2048	1.7	1908	7086	66.8	2479
StreamingLLM	4+2000	2.0	1493	2000	57.1	1311
H2O	20%	1.7	2305	7542	62.5	2660
TOVA	2048	1.7	1856	6425	62.9	2380
gemma-2-2b-it, 6382-token prompt						
vanilla (full cache)	1024	3.9	735	6382	58.7	974
μKV	1024	**9.3**	**427**	804	**58.7**	**589**
SnapKV	1024/head	4.9	913	6297	60.6	1005
Ada-KV	2048	5.0	909	6222	60.6	989
StreamingLLM	4+2000	4.9	654	2000	57.9	751
H2O	20%	3.9	1017	6382	56.0	1071
TOVA	2048	3.9	1022	6060	56.0	1063

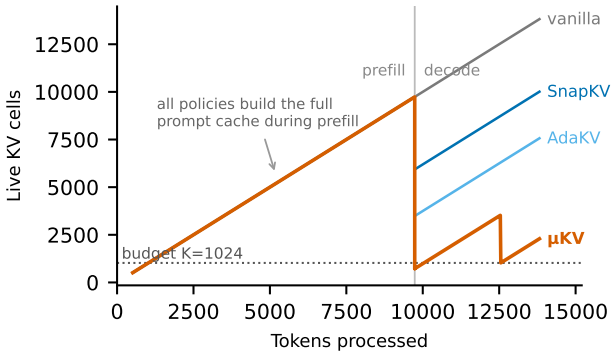

Figure 2: Live KV cells through prefill and decode on Llama-3.2-1B, phone CPU. Every policy builds the full prompt cache. Neither per-head evictor returns to its budget; μKV compacts to 723 cells and its re-eviction holds the cache near 2K.

times reduction, and on Bonsai-8B 1.0 to 1.6, against the 5 to 10 times their papers report. At their own published budgets the gap widens. On the phone CPU, over the same prompts, SnapKV at 2048 per head keeps 94% of the cache and H2O at 20% of the prompt keeps 92%. TOVA and Ada-KV at 2048 keep 77% and 71%. μKV keeps 14%. They pay the FA-off capture as well. They run at 5.8 to 6.8 tok/s and 818 to 1094 mWh against vanilla's 1033, a spread that straddles the full cache rather than beating it. StreamingLLM is the control for retention: it compacts to 777 cells, fewer than μKV, yet runs at 6.6 tok/s. Retention does not predict speed. The kernel path does. One caveat is ours. This campaign ran StreamingLLM without the fused-kernel flag, and on the GPU, where we re-ran it correctly, it reaches parity with μKV (Table 2). The CPU row understates it. Quality is unchanged. Teacher-forced perplexity on a WikiText slice verified disjoint from the prompt

Table 2: Adreno 840, Llama-3.2-1B, ctx 16384, 9737-token prompt, 4096 generated, f16 KV, cooled per cell, median where $n > 1$. Rows group by how a policy selects. The n=3 rows are one interleaved pinned campaign; the n=1 rows are the matched-budget campaign, whose own vanilla agrees within 0.5%.

Policy	n	tok/s	dec ×	Cells	% cache
vanilla (no eviction)	3	24.30	1.00	9741	100
Sequence-level: the budget frees cells, the kernel survives					
μKV, K=1024	3	**29.02**	**1.19**	**723**	**7.4**
StreamingLLM, own budget	3	29.03	1.19	2005	20.6
Per-head: the budget frees nothing, the kernel is lost					
SnapKV	1	4.76	**0.20**	6592	**67.7**
H2O	1	3.58	**0.15**	6110	**62.7**
TOVA	1	4.34	**0.18**	4125	**42.4**
Ada-KV	1	2.86	**0.12**	3519	**36.1**
Controls that isolate one property each					
μKV, compaction off	3	25.61	1.05	723	7.4
StreamingLLM, forced off the kernel	3	5.81	0.24	777	8.0

gives 8.198 against 8.023 (vanilla against μKV) on Llama-3.2-1B. It gives 3.983 against 3.938 on Phi-3-mini and 6.717 against 6.736 on Bonsai-8B. μKV had evicted 92% of the prompt cache. Weight compression shrinks the static term of per-token traffic; eviction shrinks the growing one. On 1-bit Bonsai-8B (lower block) μKV gives 4.5 times the throughput, 60% less energy and a DDR peak 9.7 ℃ cooler, with no change to its parameters. The lower two blocks run the same text on Phi-3-mini and gemma-2-2b with every baseline at its published budget. There μKV decodes 4.5 and 2.4 times faster than the full cache with 55% and 40% less energy, while the per-head and per-layer evictors keep 85 to 100% of the cache, and StreamingLLM 27 to 31%.

Mobile GPU.. Table 2 groups the Adreno results by how each policy selects, because that is what predicts the outcome. Sequence-level policies free cells and keep the fused kernel. μKV decodes at 1.19 times the full cache on 7.4% of the cells. StreamingLLM at its own documented budget of 4 sinks plus a 2000-token window reaches the same 1.19 on 20.6%. StreamingLLM is not our design, and it is the strongest evidence we have that the mechanism is real rather than an artifact of our policy. Per-head policies do the opposite. At a nominal budget of 1024 they hold 36 to 68% of the cache and decode at 0.12 to 0.20 times the full cache, so on this phone evicting is slower than not evicting.

Two controls separate the two properties. Disabling compaction in μKV leaves the same 723 selected cells but drops 1.19 to 1.05, so the throughput comes from compaction, not from selection. Forcing StreamingLLM off the fused kernel leaves its 777 cells but drops it to 0.24, so the kernel path and not the retention ratio sets the speed. That forced configuration is how our own first harness ran StreamingLLM, and it was wrong: the reference implementation concatenates survivors, so compaction is intrinsic to the policy. We report both cells rather than only the corrected one.

Speed is therefore not what separates μKV from StreamingLLM. What they keep is. StreamingLLM needs 2005 cells to μKV's 723 for the same throughput, and it answers 3 of 14 needles against μKV's 13 (Table 3), because a blind recent window discards the middle of the prompt. Scoring inside the prefill graph buys content awareness at 1.9% of prefill time.

Table 3: Quality, both benchmarks on the phone. Needle: hits out of 14 depths per model, and the mean live cache after prefill. LongBench: Llama-3.2-1B, 103 prompts, each policy at its own budget.

	Needle: hits / 14, and cells attended					LongBench	
Policy	Phi-3	Llama	Gemma	Bonsai	cells	F1	cache
vanilla	14	13	11	14	4858	36.9	100%
μKV	**14**	**13**	**11**	**14**	**774**	**37.0**	**14%**
SnapKV	14	12	11	14	4237	38.0	94%
Ada-KV	14	12	11	14	3361	37.8	71%
TOVA	14	12	11	14	3670	37.4	77%
H2O	14	8	10	13	4212	36.2	92%
StreamingLLM	**3**	**3**	**2**	**3**	890	35.0	33%

Phi-3-mini shows the engine's limits. The state-API round trip needs a second full context and the OS kills it at 16K, while in-place compaction runs there and reaches 2.64 times the full cache on 7.9% of the cells. Phi-3 also needs a driver gate: the FlashAttention-off capture is numerically broken at head dimension 96, and uncorrected runs decode at full speed while emitting 18 to 44% corrupted tokens. The gate promotes SnapKV and Ada-KV to the in-graph side node and refuses H2O and TOVA. The two promoted evictors then reach 1.15 and 1.81 times the full cache while holding 97% and 39% of it, so the gap survives the correction. Quantized KV is numerically invalid on this driver, so every GPU row is f16.

Is this one engine? The coupling lives in the memory layout, not in our code. `llama.cpp` holds one contiguous cell array per layer. Paged engines are no different where it matters: a vLLM block has shape [blocks, kv heads, head size, block size] [18], so one block carries every head for a span of positions and is returned only when all of them are finished with it. MLC-LLM [19] pages the same way, and ExecuTorch and the vendor SDKs use static position-indexed tensors. In each of them a per-head budget frees memory only where every head agrees. KV-Compress [17] reports the same limitation independently, stating that per-head eviction "adds fragmentation and cannot realize the theoretical compression rates in physical memory," and fixes it on the server by paging per head inside PagedAttention. That fix needs a per-head block table and a custom kernel. A phone runtime has neither, and a mobile GPU driver makes both expensive, which is the point: the gap is a property of how KV memory is laid out on devices.

Retrieval and long-context quality. A fact is buried at one of seven depths in 4K or 8K tokens of filler. The 392 cells cover seven policies, four models, seven depths and two contexts (Table 3, left). μKV matches the full cache on every model while attending to 774 cells, against 3361 to 4237 for the score-reading evictors, which is 69 to 87% of the full cache. StreamingLLM fails retrieval everywhere; its window evicts exactly the middle of the context. Where every policy misses a depth, on Gemma-2B and Llama-1B at 8K, the full cache misses it too. This workload generates 64 tokens, so it is prefill-dominated and says nothing useful about speed or energy; those are the 4096-token workload of Table 1. LongBench (Table 3, right) runs on the phone CPU with Llama-3.2-1B, on 103 prompts, each policy at its own published budget. μKV scores

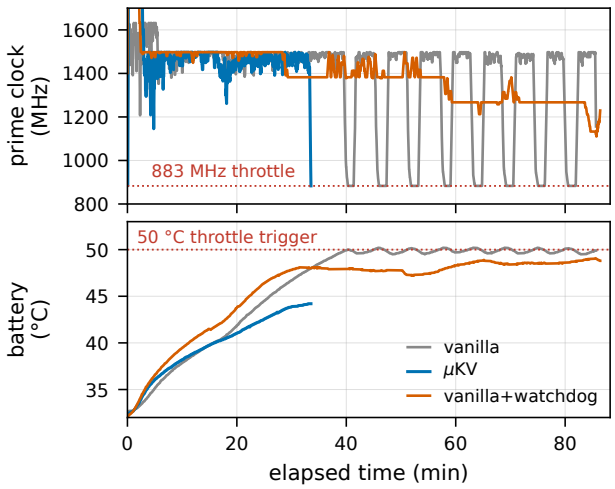

Figure 3: Bonsai-8B on the CPU from a cold start: prime clock and battery for the full cache, μKV, and the full cache with the watchdog (ablation).

37.0 against the full cache's 36.9 while holding 14% of the cache. SnapKV, Ada-KV and TOVA gain 0.6 to 1.1 F1 while holding 71 to 94%, and StreamingLLM loses 1.9 at 33%. What separates the policies is the cache each one frees, not the budget it asks for. A four-model, five-task sweep on a desktop GPU gives the same ordering at 12 to 13% of the cache. There μKV lands within 0.1 F1 of the full cache on Llama-3.2-1B and Phi-3-mini, and 2.8 and 3.7 below it on gemma-2-2b and Bonsai-8B.

Sustained heat and the watchdog. Figure 3 shows the workload that reaches the cliff. The full cache on Bonsai-8B runs 85.6 minutes and drives the battery to 50.2 ℃. At 50 ℃ the vendor deep-throttles both clusters, the prime cores to 883 MHz, in a sawtooth of 17 dips that spends 923 s at 883 MHz in total. μKV finishes in 33.2 minutes at a 44.2 ℃ battery peak, before the trigger. The watchdog belongs to μKV and is never given to a baseline; the full-cache trace with it is an ablation, since μKV finishes before the cliff. Given it, the full cache's battery rise flattens from 0.48 to 0.05 ℃/min and settles at 49.1 ℃. The prime cores never reach 883 MHz. This decode is bandwidth-bound, so the throttle costs no speed (1.0 tok/s either way): the watchdog buys sustainability, and μKV buys speed. On the GPU, three cooled cells per arm of the 4096-token request give 1289 against 1390 J and a DDR peak of 75 against 90 ℃ for 1% more time. We also tied the cache budget itself to DDR temperature, a four-tier ladder over $K \in \{256, \ldots, 1024\}$. The result is negative. Halving K under co-load left peak DDR unchanged, and on a cool phone a smaller cache runs hotter at peak, because the saved bandwidth converts to speed and power. Cache size controls throughput, time and energy; the clock caps heat.

Scope. All measurements are on one OnePlus 15. The ladder thresholds are device-specific; the calibration protocol generalizes. Energy is total-device energy from the rail and the coulomb counter, so absolute joules hold within about 10% while orderings hold. Eviction still costs verbatim recall of the spans it drops, even though prediction and retrieval are preserved. The 1-bit kernels return non-finite logits on Adreno, so Bonsai's phone results are CPU results.

5 RELATED WORK

Per-head budget allocation is Ada-KV's idea [6]; we credit it and do not reprove its bound. CriticalKV [7], DefensiveKV [8] and AhaKV [9] refine the score. R-KV [22] and DynamicKV [23] argue for task-aware budgets, as our demand-based sizing does. KIVI [5] and KVzip [15] are orthogonal. None reports a temperature, a watt or a resident set on a phone. Among mobile systems, PowerInfer-2 [12] and LLM in a Flash [13] optimize weight access. KVSwap swaps the cache to disk on a Jetson, and DynaKV offloads to UFS on the CPU. PerCache [16] shortcuts whole passes at the application level and composes with μKV. MLC-LLM [19], ExecuTorch and the vendor SDKs use the unmodified cache. Verified from their evaluation sections: the policies that evict run on servers or Jetson boards. The systems that run on phones keep every token. The studies that use a phone GPU apply no cache management. μKV occupies the empty cell. On control, zTT [20] and FUSE [21] act on frequency alone. We know of no prior system that reads attention-internal state and battery and thermal sensors in one decode loop on a phone.

6 CONCLUSION

μKV makes attention-driven KV eviction work on a phone by respecting the engine. Scores come from inside the FlashAttention graph. One keep set is what the shared cell array can free. Compaction never needs a second context. On a OnePlus 15 the result is 4.8 times the full cache's decode throughput on the CPU, at 7% of the cells and 62% less energy, and 1.19 times on the GPU. Retrieval matches the full cache and LongBench nearly does. The cache is the phone's efficiency actuator and the clock its temperature actuator.

The finding we would most like the community to take is smaller than our system. A per-head budget that looks good on a server is a full cache in disguise on the device. Selection ratios are not compression, and a paper that reports only the ratio has not shown that anything was freed. On-device work should report cells that the allocator actually reclaimed, on the engine that will ship, and should say which kernel the policy leaves behind. Our own harness failed that test on StreamingLLM before we caught it.

Open problems. Three follow directly. First, eviction is destructive: μKV cannot recall a span it dropped. The scores it already computes are a placement signal, so the natural next step is a third tier between keeping and discarding, parking mid-scored cells in NAND and reloading on demand. Reload cost is not the obstacle at 23 MB for 721 cells; the obstacle is that no mobile engine can shrink and regrow its KV pool, so parking frees nothing today. Second, nobody has measured what that parking costs in flash write energy and wear, which is the number that decides whether the tier is worth having on a battery. Third, the ladders here are calibrated to one SoC. What generalizes is the protocol of anchoring a ladder above a measured vendor trigger, not the thresholds.

REFERENCES

[1] Z. Zhang et al. "H2O: Heavy-Hitter Oracle for Efficient Generative Inference of LLMs." NeurIPS, 2023.

[2] G. Xiao et al. "Efficient Streaming Language Models with Attention Sinks." ICLR, 2024.

[3] M. Oren et al. "Transformers are Multi-State RNNs." arXiv:2401.06104, 2024.

[4] Y. Li et al. "SnapKV: LLM Knows What You're Looking For Before Generation." NeurIPS, 2024.

[5] Z. Liu et al. "KIVI: Tuning-Free Asymmetric 2bit Quantization for KV Cache." arXiv:2402.02750, 2024.

[6] Y. Feng et al. "Ada-KV: Optimizing KV Cache Eviction by Adaptive Budget Allocation." NeurIPS, 2025.

[7] Y. Feng et al. "CriticalKV: A Perturbation Perspective on KV Cache Eviction." ICML, 2026.

[8] Y. Feng et al. "DefensiveKV: Worst-Case Risk Control for KV Cache Eviction." ICLR, 2026.

[9] Y. Gu et al. "AhaKV: Adaptive Holistic Attention-Driven KV Cache Eviction." arXiv:2506.03762, 2025.

[10] T. Dao et al. "FlashAttention: Fast and Memory-Efficient Exact Attention." NeurIPS, 2022.

[11] T. Dao. "FlashAttention-2: Faster Attention with Better Parallelism." arXiv:2307.08691, 2023.

[12] Y. Song et al. "PowerInfer-2: Fast LLM Inference on a Smartphone." arXiv:2406.06282, 2024.

[13] K. Alizadeh et al. "LLM in a Flash: Efficient Inference with Limited Memory." arXiv:2312.11514, 2023.

[14] G. Gerganov. "llama.cpp." https://github.com/ggerganov/llama.cpp.

[15] J.-H. Kim et al. "KVzip: Query-Agnostic KV Cache Compression with Context Reconstruction." NeurIPS, 2025.

[16] K. Liu et al. "PerCache: Predictive Hierarchical Cache for RAG Applications on Mobile Devices." arXiv:2601.11553, 2025.

[17] I. Rehg. "KV-Compress: Paged KV-Cache Compression with Variable Compression Rates per Attention Head." arXiv:2410.00161, 2024.

[18] W. Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention." SOSP, 2023.

[19] MLC Team. "MLC-LLM: Universal LLM Deployment." https://github.com/mlc-ai/mlc-llm.

[20] S. Kim et al. "zTT: Learning-Based DVFS with Zero Thermal Throttling for Mobile Devices." MobiSys, 2021.

[21] S. Liu et al. "FUSE: Fine-Grained Power Management for Mobile Devices." SenSys, 2022.

[22] X. Cai et al. "R-KV: Redundancy-aware KV Cache Compression for Reasoning Models." arXiv:2505.24133, 2025.

[23] X. Zhou et al. "DynamicKV: Task-Aware Adaptive KV Cache Compression for Long Context LLMs." arXiv:2412.14838, 2024.

[24] PrismML. "Bonsai: Commercially Viable 1-bit LLMs." 2026. https://prismml.com/news/bonsai-8b.

[25] H. Hoffmann. "JouleGuard: Energy Guarantees for Approximate Applications." SOSP, 2015.

[26] D. H. K. Kim, C. Imes, and H. Hoffmann. "Racing and Pacing to Idle: Theoretical and Empirical Analysis of Energy Optimization Heuristics." CPSNA, 2015.

[27] B. Zou et al. "EnerInfer: Energy-Aware On-Device LLM Inference." arXiv:2606.23001, 2026.

[28] G. Cai et al. "Is Your NPU Ready for LLMs? A Power Benchmark of On-Device Inference." arXiv:2607.05475, 2026.

[29] Z. Zhang et al. "Dissecting the Impact of Mobile DVFS Governors on LLM Inference Performance and Energy Efficiency." arXiv:2507.02135, 2025.